

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и сетевые технологии**

Направление подготовки (специальность) **09.03.03 Прикладная информатика**

ПРАКТИЧЕСКАЯ РАБОТА № 1

По дисциплине «Алгоритмы и структуры данных»

Тема: Жадные алгоритмы. Динамическое программирование.
Вариант 1.

Выполнила	Борзова Д. М.; К3140.
Проверила	Ромакина О. М.
Дата	_____

Санкт-Петербург 2026

Оглавление

Оглавление	2
Задачи по варианту	4
1 задача. Максимальная стоимость добычи	4
Описание задания к задаче	4
Исходный код и описание решения	4
Описание проведённых тестов	5
Вывод по задаче	7
3 задача. Максимальный доход от рекламы	8
Описание задания к задаче	8
Исходный код и описание решения	8
Описание проведённых тестов	9
Вывод по задаче	10
8 задача. Расписание лекций	11
Описание задания к задаче	11
Исходный код и описание решения	11
Описание проведённых тестов	12
Вывод по задаче	13
14 задача. Максимальное значение арифметического выражения	15
Описание задания к задаче	15
Исходный код и описание решения	15
Описание проведённых тестов	17
Вывод по задаче	18
17 задача. Ход конем.....	19
Описание задания к задаче	19
Исходный код и описание решения	19
Описание проведённых тестов	20
Вывод по задаче	21
Дополнительные задачи.	22
11 задача. Максимальное количество золота (1 балл, с учётом штрафа 0,5 балла)	22
Описание задания к задаче	22
Исходный код и описание решения	22
Описание проведённых тестов	23
Вывод по задаче	24
13 задача. Сувениры (1.5 балла, с учётом штрафа 1 балл)	25

Описание задания к задаче	25
Исходный код и описание решения	25
Описание проведённых тестов	27
Вывод по задаче	28
16 задача. Продавец (2 балла, с учётом штрафа 1,5 балла)	29
Описание задания к задаче	29
Исходный код и описание решения	29
Описание проведённых тестов	31
Вывод по задаче	32
19 задача. Произведение матриц (3 балла, с учётом штрафа 2,5 балла).....	33
Описание задания к задаче	33
Исходный код и описание решения	33
Описание проведённых тестов	35
Вывод по задаче	35
22 задача. Симпатичные узоры (4 балла, с учётом штрафа 3,5 балла)	36
Описание задания к задаче	36
Исходный код и описание решения	36
Описание проведённых тестов	38
Вывод по задаче	39
Вывод по проделанной работе	40

Задачи по варианту

1 задача. Максимальная стоимость добычи

Описание задания к задаче

Вор находит гораздо больше добычи, чем может поместиться в его сумке. Помогите ему найти самую ценную комбинацию предметов, предполагая, что любая часть предмета добычи может быть помещена в его сумку.

Цель - реализовать алгоритм для задачи о дробном рюкзаке.

- **Формат ввода / входного файла (input.txt).** В первой строке входных данных задано целое число n - количество предметов, и W - вместимость сумки. Следующие n строк определяют значения веса и стоимости предметов. В i -ой строке содержатся целые числа p_i и w_i - стоимость и вес i -го предмета, соответственно.
- **Ограничения на входные данные.** $1 \leq n \leq 10^3$, $0 \leq W \leq 2 \cdot 10^6$, $0 \leq p_i \leq 2 \cdot 10^6$, $0 \leq w_i \leq 2 \cdot 10^6$ для всех $1 \leq i \leq n$. Все числа - целые.
- **Формат вывода / выходного файла (output.txt).** Выведите максимальное значение стоимости долей предметов, которые помещаются в сумку. Абсолютная погрешность между ответом вашей программы и оптимальным значением должно быть не более 10^{-3} . Для этого выведите свой ответ как минимум с четырьмя знаками после запятой (иначе ваш ответ, хотя и будет рассчитан правильно, может оказаться неверным из-за проблем с округлением).
- Ограничение по времени. 2 сек.

Исходный код и описание решения

```
def solve_fractional_knapsack():
    with open('input.txt', 'r') as f:
        first_line = f.readline().strip()
        while first_line == '':
            first_line = f.readline().strip()
        n, capacity = map(int, first_line.split())

        items = []
        for _ in range(n):
            line = f.readline().strip()
            while line == '':
                line = f.readline().strip()
            price, weight = map(int, line.split())
            items.append((price, weight))
```

```

    items.sort(key=lambda x: x[0] / x[1] if x[1] > 0 else float('inf'),
reverse=True)

    total_value = 0.0
    remaining_capacity = capacity

    for price, weight in items:
        if weight == 0:
            total_value += price
            continue
        if remaining_capacity <= 0:
            break
        if weight <= remaining_capacity:
            total_value += price
            remaining_capacity -= weight
        else:
            fraction = remaining_capacity / weight
            total_value += price * fraction
            remaining_capacity = 0

    with open('output.txt', 'w') as f:
        f.write(f"{total_value:.4f}")

if __name__ == "__main__":
    solve_fractional_knapsack()

```

Сначала читаем из input.txt число предметов n и вместимость сумки, затем построчно считываем пары «стоимость - вес» в список. Сортируем список по убыванию отношения стоимости к весу. Для предметов с нулевым весом отношение не определено, поэтому они считаются «бесплатными» и ставятся в начало.

Далее проходим по отсортированному списку, поддерживая переменную, в которой храним, сколько места ещё осталось в сумке. Если предмет с нулевым весом - сразу добавляем его стоимость к ответу. Если предмет помещается целиком - берём его полностью и уменьшаем остаток. Иначе берём только ту часть, которая влезает: стоимость умножаем на отношение оставшегося места к весу предмета, обнуляем остаток и прерываем цикл.

Итоговое значение записываем в output.txt с четырьмя знаками после запятой. Сложность алгоритма - $O(n \log n)$ за счёт сортировки.

Описание проведённых тестов

input.txt			
1	3 50		
2	60 20		
3	100 50		
4	120 30		
5			

output.txt	
1	180.0000

input.txt			
1	1 10		
2	500 30		
3			

output.txt	
1	166.6667

```

7 ▶ def test_minimal():  @ MarileeMorte
8     check( task_num: 1, input_data: "1 0\n0 0\n", expected: "0.0000")
9
10
11 ▶ def test_example_1():  @ MarileeMorte
12     check( task_num: 1, input_data: "3 50\n60 20\n100 50\n120 30\n", expected: "180.0000")
13
14
15 ▶ def test_example_2():  @ MarileeMorte
16     check( task_num: 1, input_data: "1 10\n500 30\n", expected: "166.6667")
17
18
19 ▶ def test_maximal():  @ MarileeMorte
20     n = 1000
21     capacity = 2 * 10 ** 6
22     lines = [f"{n} {capacity}"]
23     for _ in range(n):
24         lines.append(f"{2 * 10 ** 6} {2 * 10 ** 6}")
25     check( task_num: 1, "\n".join(lines) + "\n", expected: "2000000.0000")

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.072s	memory=56.0 KB
Первый пример из задачи	time=0.082s	memory=160.0 KB
Второй пример из задачи	time=0.092s	memory=112.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.100s	memory=228.0 KB

Вывод по задаче

Был реализован жадный алгоритм для задачи о дробном рюкзаке: предметы сортируются по удельной стоимости, после чего набираются, пока позволяет вместимость. Алгоритм корректно обрабатывает нулевые веса и частичный набор последнего предмета.

3 задача. Максимальный доход от рекламы

Описание задания к задаче

У вас есть n объявлений для размещения на популярной интернет-странице. Для каждого объявления вы знаете, сколько рекламодатель готов платить за один клик по этому объявлению. Вы настроили n слотов на своей странице и оценили ожидаемое количество кликов в день для каждого слота. Теперь ваша цель - распределить рекламу по слотам, чтобы максимизировать общий доход.

- **Постановка задачи.** Даны две последовательности $a_1, a_2, \dots, a_n$ (a_i - прибыль за клик по i -му объявлению) и $b_1, b_2, \dots, b_n$ (b_i - среднее количество кликов в день i -го слота), нужно разбить их на n пар (a_i, b_j) так, чтобы сумма их произведений была максимальной.
- **Формат ввода / входного файла (input.txt).** В первой строке содержится целое число n , во второй - последовательность целых чисел $a_1, a_2, \dots, a_n$, в третьей - последовательность целых чисел $b_1, b_2, \dots, b_n$.
- **Ограничения на входные данные.** $1 \leq n \leq 10^3$, $-10^5 \leq a_i, b_i \leq 10^5$, для всех $1 \leq i \leq n$.
- **Формат вывода / выходного файла (output.txt).** Выведите максимальное значение $\sum_{i=1}^n a_i c_i$, где $c_1, c_2, \dots, c_n$ является перестановкой $b_1, b_2, \dots, b_n$.
- Ограничение по времени. 2 сек.

Исходный код и описание решения

```
def solve_max_advertising_revenue():
    with open('input.txt', 'r') as f:
        n = int(f.readline().strip())

        a = []
        while len(a) < n:
            a.extend(map(int, f.readline().strip().split()))

        b = []
        while len(b) < n:
            b.extend(map(int, f.readline().strip().split()))

        a.sort()
        b.sort()

        total_revenue = sum(a[i] * b[i] for i in range(n))

        with open('output.txt', 'w') as f:
            f.write(str(total_revenue))
```

```
if __name__ == "__main__":
    solve_max_advertising_revenue()
```

По неравенству перестановки максимальная сумма попарных произведений достигается, когда обе последовательности упорядочены одинаково. Поэтому алгоритм прост: читаем из input.txt число n , затем обе последовательности, сортируем их по возрастанию и вычисляем сумму произведений соответствующих элементов.

Такое сопоставление гарантирует, что наибольшее значение первой последовательности умножается на наибольшее значение второй, а наименьшее - на наименьшее. Именно это и даёт максимум: если бы мы сопоставили их иначе, поменяв местами два слагаемых, сумма бы только уменьшилась. Результат записываем в output.txt.

Сложность - $O(n \log n)$ из-за сортировки, при $n \leq 10^3$ работает мгновенно.

Описание проведённых тестов

input.txt		output.txt	
1	1	1	897
2	23		
3	39		
4			

input.txt		output.txt	
1	3	1	23
2	1 3 -5		
3	-2 4 1		
4			

```

7 ▶ def test_minimal():  Ⓜ MarieeMorte
8     check( task_num: 3, input_data: "1\n5\n7\n", expected: "35")
9
10
11 ▶ def test_example_1():  Ⓜ MarieeMorte
12     check( task_num: 3, input_data: "1\n23\n39\n", expected: "897")
13
14
15 ▶ def test_example_2():  Ⓜ MarieeMorte
16     check( task_num: 3, input_data: "3\n1 3 -5\n-2 4 1\n", expected: "23")
17
18
19 ▶ def test_maximal():  Ⓜ MarieeMorte
20     n = 1000
21     a = " ".join(["100000"] * n)
22     b = " ".join(["100000"] * n)
23     expected = str(100000 * 100000 * n)
24     check( task_num: 3, input_data: f"{n}\n{a}\n{b}\n", expected)

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.072s	memory=20.0 KB
Первый пример из задачи	time=0.084s	memory=20.0 KB
Второй пример из задачи	time=0.087s	memory=116.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.097s	memory=228.0 KB

Вывод по задаче

Был реализован алгоритм максимизации скалярного произведения на основе неравенства перестановки: обе последовательности сортируются, после чего вычисляется сумма попарных произведений. Решение корректно работает и с отрицательными значениями.

8 задача. Расписание лекций

Описание задания к задаче

- **Постановка задачи.** Вы наверно знаете, что в ИТМО лекции читают одни из лучших преподаватели мира. К сожалению, лекционных аудиторий у нас не так уж и много, особенно на Биржевой, поэтому каждый преподаватель составил список лекций, которые он хочет прочитать студентам. Чтобы студенты, в начале февраля, увидели расписание лекций, необходимо его составить прямо сейчас. И без вас нам здесь не справиться. У нас есть список заявок от преподавателей на лекции для одной из аудиторий. Каждая заявка представлена в виде временного интервала $[s_i, f_i)$ - время начала и конца лекции. Лекция считается открытым интервалом, то есть какая-то лекция может начаться в момент окончания другой, без перерыва. Необходимо выбрать из этих заявок такое подмножество, чтобы суммарно выполнить максимальное количество заявок. Учтите, что одновременно в лекционной аудитории, конечно же, может читаться лишь одна лекция.
- **Формат ввода / входного файла (input.txt).** В первой строке вводится натуральное число N - общее количество заявок на лекции. Затем вводится N строк с описаниями заявок - по два числа в каждом s_i и f_i для каждой лекции i . Гарантируется, что $s_i < f_i$. Время начала и окончания лекции - натуральные числа, не превышают 1440 (в минутах с начала суток).
- **Ограничения на входные данные.** $1 \leq N \leq 1000$, $1 \leq s_i, f_i \leq 1440$
- **Формат вывода / выходного файла (output.txt).** Выведите одно число – максимальное количество заявок на проведение лекций, которые можно выполнить.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.

Исходный код и описание решения

```
def solve_lecture_scheduling():
    with open('input.txt', 'r') as f:
        n = int(f.readline().strip())

        lectures = []
        for _ in range(n):
            start, end = map(int, f.readline().strip().split())
```

```

        lectures.append((start, end))

    lectures.sort(key=lambda x: x[1])

    count = 0
    last_end_time = 0

    for start, end in lectures:
        if start >= last_end_time:
            count += 1
            last_end_time = end

    with open('output.txt', 'w') as f:
        f.write(str(count))

if __name__ == "__main__":
    solve_lecture_scheduling()

```

Ключевая идея: нужно всегда выбирать лекцию, которая заканчивается раньше всех - тогда остаётся максимум времени для последующих.

Читаем из input.txt количество заявок n , затем n пар «время начала - время окончания».

Сортируем список лекций по времени окончания.

Проходим по отсортированному списку, поддерживая переменную, в которой храним момент окончания последней выбранной лекции. Если текущая лекция начинается не раньше этого момента, то её можно взять: увеличиваем счётчик и обновляем время окончания. Иначе пропускаем. Важно, что условие именно «не меньше», а не «строго больше»: в задаче лекции считаются открытыми интервалами, поэтому лекция $[1, 5)$ и лекция $[5, 10)$ совместимы.

В конце записываем число выбранных лекций в output.txt. Сложность - $O(n \log n)$ из-за сортировки.

Описание проведённых тестов

input.txt		output.txt	
1	1	1	1
2	5 10		
3			

input.txt			
1	3		
2	1 5		
3	2 3		
4	3 4		
5		1	2

```

6  ▶ def test_minimal():  Ⓜ MarieeMorte
7      check( task_num: 8, input_data: "1\n5 10\n", expected: "1")
8
9
10 ▶ def test_example_1():  Ⓜ MarieeMorte
11     check( task_num: 8, input_data: "1\n5 10\n", expected: "1")
12
13
14 ▶ def test_example_2():  Ⓜ MarieeMorte
15     check( task_num: 8, input_data: "3\n1 5\n2 3\n3 4\n", expected: "2")
16
17
18 ▶ def test_maximal():  Ⓜ MarieeMorte
19     n = 1000
20     lines = [str(n)] + [f"{i + 1} {i + 2}" for i in range(n)]
21     check( task_num: 8, "\n".join(lines) + "\n", str(n))
22

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.073s	memory=4.0 KB
Первый пример из задачи	time=0.075s	memory=60.0 KB
Второй пример из задачи	time=0.086s	memory=136.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.095s	memory=164.0 KB

Вывод по задаче

Был реализован жадный алгоритм выбора максимального числа непересекающихся интервалов. Сортировка лекций по времени окончания и последовательный набор

совместимых заявок дают оптимальный ответ, при этом корректно обрабатываются стыкующиеся интервалы.

14 задача. Максимальное значение арифметического выражения

Описание задания к задаче

В этой задаче ваша цель - добавить скобки к заданному арифметическому выражению, чтобы максимизировать его значение.

$$\max(5 - 8 + 7 \times 4 - 8 + 9) = ?$$

- **Постановка задачи.** Найдите максимальное значение арифметического выражения, указав порядок применения его арифметических операций с помощью дополнительных скобок.
- **Формат ввода / входного файла (input.txt).** Единственная строка входных данных содержит строку s длины $2n + 1$ для некоторого n с символами $s_0, s_1, \dots, s_{2n}$. Каждый символ в четной позиции s является цифрой (то есть целым числом от 0 до 9), а каждый символ в нечетной позиции является одной из трех операций из $+, -, *$
- **Ограничения на входные данные.** $0 \leq n \leq 14$ (следовательно, строка содержит не более 29 символов).
- **Формат вывода / выходного файла (output.txt).** Выведите максимально возможное значение заданного арифметического выражения среди различных порядков применения арифметических операций.
- Ограничение по времени. 5 сек.

Исходный код и описание решения

```
def solve_max_arithmetic_expression():
    with open('input.txt', 'r') as f:
        expression = f.readline().strip()

    numbers = []
    operations = []

    for i, char in enumerate(expression):
        if i % 2 == 0:
            numbers.append(int(char))
        else:
            operations.append(char)

    n = len(numbers)

    dp_min = [[float('inf')] * n for _ in range(n)]
    dp_max = [[-float('inf')] * n for _ in range(n)]

    for i in range(n):
        dp_min[i][i] = numbers[i]
```

```

dp_max[i][i] = numbers[i]

for length in range(2, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1

        for k in range(i, j):
            op = operations[k]

            left_min = dp_min[i][k]
            left_max = dp_max[i][k]
            right_min = dp_min[k + 1][j]
            right_max = dp_max[k + 1][j]

            results = []
            if op == '+':
                results = [
                    left_min + right_min,
                    left_min + right_max,
                    left_max + right_min,
                    left_max + right_max
                ]
            elif op == '-':
                results = [
                    left_min - right_min,
                    left_min - right_max,
                    left_max - right_min,
                    left_max - right_max
                ]
            elif op == '*':
                results = [
                    left_min * right_min,
                    left_min * right_max,
                    left_max * right_min,
                    left_max * right_max
                ]

            dp_min[i][j] = min(dp_min[i][j], min(results))
            dp_max[i][j] = max(dp_max[i][j], max(results))

result = dp_max[0][n - 1]

with open('output.txt', 'w') as f:
    f.write(str(result))

if __name__ == "__main__":
    solve_max_arithmetic_expression()

```

Разбираем входную строку на два массива: цифры на чётных позициях и знаки операций на нечётных. Обозначим n - количество чисел.

Используем динамическое программирование по подотрезкам. Заводим два двумерных массива: для минимального и для максимального значения, которое можно получить из подвыражения с i -го по j -е число включительно. Хранить минимум необходимо из-за операций вычитания и умножения: при умножении на отрицательное число «минимум» может дать «максимум», и наоборот.

База: для одного числа обе таблицы содержат само это число. Далее перебираем длину подотрезка от 2 до n и для каждой пары (i, j) перебираем позицию разбиения k от i до $j-1$. Для каждого k пересчитываем четыре комбинации (минимум с минимумом, минимум с максимумом, максимум с минимумом, максимум с максимумом), применяем операцию и обновляем оба массива.

Ответ - максимальное значение для всего выражения. Записываем его в output.txt.

Сложность $O(n^3)$, при $n \leq 15$ работает мгновенно.

Описание проведённых тестов

input.txt		output.txt	
1	1+5	1	6
2			

input.txt		output.txt	
1	5-8+7*4-8+9	1	200
2			

```
6 ▶ def test_minimal():  @ MarieeMorte
7     check( task_num: 14, input_data: "0\n", expected: "0")
8
9
10 ▶ def test_example_1():  @ MarieeMorte
11     check( task_num: 14, input_data: "1+5\n", expected: "6")
12
13
14 ▶ def test_example_2():  @ MarieeMorte
15     check( task_num: 14, input_data: "5-8+7*4-8+9\n", expected: "200")
16
17
18 ▶ def test_maximal_add():  @ MarieeMorte
19     expr = "+".join(["9"] * 15)
20     check( task_num: 14, expr + "\n", expected: "135")
21
22
23 ▶ def test_maximal_mul():  @ MarieeMorte
24     expr = "*".join(["9"] * 15)
25     check( task_num: 14, expr + "\n", str(9 ** 15))
```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.073s	memory=140.0 KB
Первый пример из задачи	time=0.083s	memory=240.0 KB
Второй пример из задачи	time=0.076s	memory=216.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.088s	memory=328.0 KB

Вывод по задаче

Был реализован алгоритм динамического программирования по подотрезкам. Совместное хранение минимума и максимума позволяет корректно обрабатывать вычитание и умножение, при которых знак результата может меняться.

17 задача. Ход конем

Описание задания к задаче

- **Постановка задачи.** Шахматная ассоциация решила оснастить всех своих сотрудников такими телефонными номерами, которые бы набирались на кнопочном телефоне ходом коня. Например, ходом коня набирается телефон 340-49-27. При этом телефонный номер не может начинаться ни с цифры 0, ни с цифры 8.
Напишите программу, определяющую количество телефонных номеров длины N , набираемых ходом коня. Поскольку таких номеров может быть очень много, выведите ответ по модулю 10^9 .
- **Формат ввода / входного файла (input.txt).** Во входном файле записано одно целое число N .
- **Ограничения на входные данные.** $1 \leq N \leq 1000$.
- **Формат вывода / выходного файла (output.txt).** Выведите в выходной файл искомое количество телефонных номеров по модулю 10^9 .
- Ограничение по времени. 1 сек.
- Ограничение по памяти. 256 мб.

Исходный код и описание решения

```
def solve_knight_moves():
    with open('input.txt', 'r') as f:
        n = int(f.readline().strip())

    mod = 10 ** 9

    moves = {
        0: [4, 6],
        1: [6, 8],
        2: [7, 9],
        3: [4, 8],
        4: [0, 3, 9],
        5: [],
        6: [0, 1, 7],
        7: [2, 6],
        8: [1, 3],
        9: [2, 4]
    }

    dp = [0] * 10

    for digit in range(10):
        if digit != 0 and digit != 8:
            dp[digit] = 1
```

```

for _ in range(2, n + 1):
    new_dp = [0] * 10

    for digit in range(10):
        for prev_digit in moves[digit]:
            new_dp[digit] = (new_dp[digit] + dp[prev_digit]) % mod

    dp = new_dp

result = sum(dp) % mod

with open('output.txt', 'w') as f:
    f.write(str(result))

if __name__ == "__main__":
    solve_knight_moves()

```

Сначала задаём таблицу ходов коня для каждой цифры. Цифры расположены на клавиатуре 4×3: верхний ряд 1, 2, 3, средний 4, 5, 6, нижний 7, 8, 9, и отдельно внизу 0. Например, из 0 можно попасть в 4 и 6, из 1 - в 6 и 8, из 5 - никуда. Ходы симметричны. Заводим массив длины 10, где каждое значение - количество номеров текущей длины, оканчивающихся на соответствующую цифру. База для N = 1: все цифры кроме 0 и 8 дают по одному номеру.

Для каждой следующей длины (от 2 до N) строим новый массив. Для каждой цифры перебираем все цифры, из которых можно попасть в неё ходом коня, и складываем их значения по модулю 10⁹. После N-1 итерации суммируем массив по всем цифрам - это и есть ответ. Записываем его в output.txt.

Сложность - O(N · 100), при N = 1000 это 100 тысяч операций.

Описание проведённых тестов

input.txt × <table> <tr><td>1</td><td>1</td></tr> <tr><td>2</td><td></td></tr> </table>	1	1	2		output.txt × <table> <tr><td>1</td><td>8</td></tr> <tr><td></td><td></td></tr> </table>	1	8		
1	1								
2									
1	8								
input.txt × <table> <tr><td>1</td><td>2</td></tr> <tr><td>2</td><td></td></tr> </table>	1	2	2		output.txt × <table> <tr><td>1</td><td>16</td></tr> <tr><td></td><td></td></tr> </table>	1	16		
1	2								
2									
1	16								

```

19 ▶ def test_minimal():  & MarieeMorte
20     check( task_num: 17, input_data: "1\n", expected: "8")
21
22
23 ▶ def test_example_1():  & MarieeMorte
24     check( task_num: 17, input_data: "1\n", expected: "8")
25
26
27 ▶ def test_example_2():  & MarieeMorte
28     check( task_num: 17, input_data: "2\n", expected: "16")
29
30
31 ▶ def test_maximal():  & MarieeMorte
32     output = check( task_num: 17, input_data: "1000\n", timeout=30)
33     value = int(output)
34     assert 0 <= value < MOD
35     assert value == _reference(1000)

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.075s	memory=84.0 KB
Первый пример из задачи	time=0.084s	memory=248.0 KB
Второй пример из задачи	time=0.091s	memory=292.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.091s	memory=316.0 KB

Вывод по задаче

Был реализован алгоритм динамического программирования по длине номера с предвычисленной таблицей ходов коня. Использование массива из десяти счётчиков вместо полного перебора позволяет уложиться в ограничения при $N = 1000$.

Дополнительные задачи.

11 задача. Максимальное количество золота (1 балл, с учётом штрафа 0,5 балла)

Описание задания к задаче

Вам дается набор золотых слитков, и ваша цель - набрать как можно больше золота в свою сумку. Существует только одна копия каждого слитка, и для каждого слитка вы можете либо взять его, либо нет (т.е. вы не можете взять часть слитка).

- **Постановка задачи.** Даны n золотых слитков, найдите максимальный вес золота, который поместится в сумку вместимостью W .
- **Формат ввода / входного файла (input.txt).** Первая строка входных данных содержит вместимость W сумки и количество n золотых слитков. В следующей строке записано n целых чисел $w_0, w_1, \dots, w_{n-1}$, определяющие вес золотых слитков.
- **Ограничения на входные данные.** $1 \leq W \leq 10^4$, $1 \leq n \leq 300$, $0 \leq w_0, \dots, w_{n-1} \leq 10^5$
- **Формат вывода / выходного файла (output.txt).** Выведите максимальный вес золота, который поместится в сумку вместимости W .
- Ограничение по времени. 5 сек.

Исходный код и описание решения

```
def solve_knapsack():
    with open('input.txt', 'r') as f:
        first_line = f.readline().strip()
        while first_line == '':
            first_line = f.readline().strip()
        capacity, n = map(int, first_line.split())

        weights = []
        while len(weights) < n:
            weights.extend(map(int, f.readline().strip().split()))

    dp = [0] * (capacity + 1)

    for weight in weights:
        if weight == 0:
            continue
        for w in range(capacity, weight - 1, -1):
            if dp[w - weight] + weight > dp[w]:
                dp[w] = dp[w - weight] + weight

    max_weight = dp[capacity]
```

```

with open('output.txt', 'w') as f:
    f.write(str(max_weight))

if __name__ == "__main__":
    solve_knapsack()

```

Читаем из input.txt вместимость и число слитков, затем веса всех слитков. Создаём одномерный массив длины «вместимость + 1», где каждое значение - максимальный вес золота, который можно набрать при соответствующей вместимости. Изначально все значения нули.

Для каждого слитка проходим по вместимостям в обратном порядке - от максимума вниз до веса слитка. Для каждой вместимости проверяем, станет ли лучше, если добавить этот слиток: сравниваем текущее значение с суммой значения для вместимости минус вес слитка плюс сам вес. Обратный порядок обхода критичен: он гарантирует, что один и тот же слиток не будет использован дважды.

После обработки всех слитков ответ - значение для полной вместимости. Записываем его в output.txt. Сложность - $O(n \times \text{вместимость})$, при $n = 300$ и вместимости 10^4 это около 3 миллионов операций.

Описание проведённых тестов

input.txt		output.txt	
1	10 3	1	9
2	1 4 8		
3			

```

6  ▶ def test_minimal():  Ⓜ MarieeMorte
7      check( task_num: 11, input_data: "1 1\n1\n", expected: "1")
8
9
10 ▶ def test_example():  Ⓜ MarieeMorte
11     check( task_num: 11, input_data: "10 3\n1 4 8\n", expected: "9")
12
13
14 ▶ def test_maximal():  Ⓜ MarieeMorte
15     n = 300
16     w = 10000
17     weights = " ".join(["100"] * n)
18     check( task_num: 11, input_data: f"{w} {n}\n{weights}\n", expected: "10000")
19
20
21 ▶ def test_maximal_all_too_heavy():  Ⓜ MarieeMorte
22     n = 300
23     w = 10000
24     weights = " ".join(["100000"] * n)
25     check( task_num: 11, input_data: f"{w} {n}\n{weights}\n", expected: "0")

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.075s	memory=20.0 KB
Пример из задачи	time=0.085s	memory=72.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.239s	memory=164.0 KB

Вывод по задаче

Был реализован алгоритм динамического программирования для задачи о рюкзаке 0/1.
 Использование одномерного массива с обратным порядком обхода позволило
 сэкономить память и избежать повторного использования одного слитка.

13 задача. Сувениры (1.5 балла, с учётом штрафа 1 балл)

Описание задания к задаче

Вы и двое ваших друзей только что вернулись домой после посещения разных стран.

Теперь вы хотели бы поровну разделить все сувениры, которые все трое накупили.

- **Формат ввода / входного файла (input.txt).** В первой строке дано целое число n . Во второй строке даны целые числа $v_1, v_2, \dots, v_n$, разделенные пробелами.
- **Ограничения на входные данные.** $1 \leq n \leq 20$, $1 \leq v_i \leq 30$ для всех i .
- **Формат вывода / выходного файла (output.txt).** Выведите 1, если можно разбить $v_1, v_2, \dots, v_n$ на три подмножества с одинаковыми суммами и 0 в противном случае.
- Ограничение по времени. 5 сек.

Исходный код и описание решения

```
def can_partition_into_three_equal_sums():
    with open('input.txt', 'r') as f:
        n = int(f.readline().strip())
        values = list(map(int, f.readline().strip().split()))

    total_sum = sum(values)

    if total_sum % 3 != 0:
        result = 0
    else:
        target = total_sum // 3

        subset_sum = [0] * (1 << n)

        for mask in range(1 << n):
            sum_mask = 0
            for i in range(n):
                if mask & (1 << i):
                    sum_mask += values[i]
            subset_sum[mask] = sum_mask

        dp = [False] * (1 << n)
        dp[0] = True

        for mask in range(1 << n):
            if not dp[mask]:
                continue

            for i in range(n):
                if mask & (1 << i):
                    continue

                new_mask = mask | (1 << i)
                if subset_sum[mask] + values[i] <= target:
                    dp[new_mask] = True
```

```

        result = 0
        for mask in range(1 << n):
            if subset_sum[mask] == target and dp[mask]:
                remaining_mask = ((1 << n) - 1) ^ mask
                if can_partition_into_two(remaining_mask, target, values,
n):
                    result = 1
                    break

        with open('output.txt', 'w') as f:
            f.write(str(result))

def can_partition_into_two(mask, target, values, n):
    total = 0
    for i in range(n):
        if mask & (1 << i):
            total += values[i]

    if total != 2 * target:
        return False

    sub_mask = mask
    while sub_mask:
        sum_sub = 0
        for i in range(n):
            if sub_mask & (1 << i):
                sum_sub += values[i]
        if sum_sub == target:
            remaining = mask ^ sub_mask
            sum_remaining = 0
            for i in range(n):
                if remaining & (1 << i):
                    sum_remaining += values[i]
            if sum_remaining == target:
                return True
        sub_mask = (sub_mask - 1) & mask

    return False

if __name__ == "__main__":
    can_partition_into_three_equal_sums()

```

Сначала проверяем делимость общей суммы на 3 - если не делится, ответ сразу 0. Иначе целевая сумма каждой группы равна трети от общей.

Основная идея - динамическое программирование по подмножествам через битовые маски, так как $n \leq 20$ позволяет перебрать все 2^{20} подмножеств. Строим массив, в котором для каждой маски хранится сумма элементов, попавших в неё. Для быстрого вычисления используем трюк с младшим установленным битом: сумма для маски равна сумме для маски без этого бита плюс значение соответствующего элемента.

Затем перебираем все маски, у которых сумма равна целевой. Для каждой такой маски проверяем, что дополнение можно разбить ещё на две группы по целевой сумме. Это

делается перебором подмасок дополнения - ищем подмаску с нужной суммой; если находим, значит и вторая группа автоматически имеет нужную сумму.

Если хотя бы одна такая тройка нашлась - выводим 1, иначе 0. Сложность - $O(2^n \cdot n)$, при $n = 20$ это около 20 миллионов операций.

Описание проведённых тестов

input.txt				output.txt	
1	4				
2	3 3 3 3				
3				1	0

input.txt				output.txt	
1	1				
2	40				
3				1	0

input.txt				output.txt	
1	11				
2	17 59 34 57 17 23 67 1 18 2 59				
3				1	1

input.txt				output.txt	
1	13				
2	1 2 3 4 5 5 7 7 8 10 12 19 25				
3				1	1

```

7 ▶ def test_minimal(): & MarieeMorte
8     check( task_num: 13, input_data: "1\n3\n", expected: "0")
9
10
11 ▶ def test_example_1(): & MarieeMorte
12     check( task_num: 13, input_data: "4\n3 3 3\n", expected: "0")
13
14
15 ▶ def test_example_2(): & MarieeMorte
16     check( task_num: 13, input_data: "1\n40\n", expected: "0")
17
18
19 ▶ def test_example_3(): & MarieeMorte
20     check( task_num: 13, input_data: "11\n17 59 34 57 17 23 67 1 18 2 59\n", expected: "1")
21
22
23 ▶ def test_example_4(): & MarieeMorte
24     check( task_num: 13, input_data: "13\n1 2 3 4 5 5 7 7 8 10 12 19 25\n", expected: "1")
25
26
27 ▶ def test_maximal_yes(): & MarieeMorte
28     n = 18
29     values = " ".join(["10"] * n)
30     check( task_num: 13, input_data: f"{n}\n{n}{values}\n", expected: "1")
31
32
33 ▶ def test_maximal_no(): & MarieeMorte
34     n = 20
35     values = " ".join(["30"] * n)
36     check( task_num: 13, input_data: f"{n}\n{n}{values}\n", expected: "0")

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.081s	memory=60.0 KB
Первый пример из задачи	time=0.096s	memory=124.0 KB
Второй пример из задачи	time=0.092s	memory=208.0 KB
Третий пример из задачи	time=0.088s	memory=180.0 KB
Четвёртый пример из задачи	time=0.099s	memory=80.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=1.558s	memory=37.9 MB

Вывод по задаче

Был реализован алгоритм перебора подмножеств через битовые маски. Быстрое вычисление сумм через младший установленный бит и проверка дополнения на разбиение позволяют определить возможность разделения на три равные группы при $n \leq 20$.

16 задача. Продавец (2 балла, с учётом штрафа 1,5 балла)

Описание задания к задаче

- **Постановка задачи.** Продавец техники хочет объехать n городов, посетив каждый из них ровно один раз. Помогите ему найти кратчайший путь.
- **Формат ввода / входного файла (input.txt).** Первая строка входного файла содержит натуральное число n – количество городов. Следующие n строк содержат по n чисел – длины путей между городами. В i -й строке j -е число – $a_{i,j}$ – это расстояние между городами i и j .
- **Ограничения на входные данные.** $1 \leq n \leq 13$, $0 \leq a_{i,j} \leq 10^6$, $a_{i,j} = a_{j,i}$, $a_{i,i} = 0$.
- **Формат вывода / выходного файла (output.txt).** В первой строке выходного файла выведите длину кратчайшего пути. Во второй строке выведите n чисел – порядок, в котором нужно посетить города.
- Ограничение по времени. 1 сек.
- Ограничение по памяти. 256 мб.

Исходный код и описание решения

```
def solve_tsp():
    with open('input.txt', 'r') as f:
        n = int(f.readline().strip())

        dist = []
        for _ in range(n):
            row = list(map(int, f.readline().strip().split()))
            dist.append(row)

        inf = float('inf')
        dp = [[inf] * n for _ in range(1 << n)]
        parent = [[-1] * n for _ in range(1 << n)]

        for i in range(n):
            dp[1 << i][i] = 0

        for mask in range(1 << n):
            for last in range(n):
                if dp[mask][last] == inf:
                    continue

                for next_city in range(n):
                    if mask & (1 << next_city):
                        continue

                    new_mask = mask | (1 << next_city)
                    new_dist = dp[mask][last] + dist[last][next_city]

                    if new_dist < dp[new_mask][next_city]:
```

```

        dp[new_mask][next_city] = new_dist
        parent[new_mask][next_city] = last

    full_mask = (1 << n) - 1
    best_dist = inf
    best_last = -1

    for last in range(n):
        if dp[full_mask][last] < best_dist:
            best_dist = dp[full_mask][last]
            best_last = last

    path = []
    mask = full_mask
    last = best_last

    while last != -1:
        path.append(last + 1)
        prev = parent[mask][last]
        mask = mask ^ (1 << last)
        last = prev

    path.reverse()

    with open('output.txt', 'w') as f:
        f.write(str(best_dist) + '\n')
        f.write(' '.join(map(str, path)))

    if len(path) > 0:
        f.write(' ')

if __name__ == "__main__":
    solve_tsp()

```

Это задача о кратчайшем гамильтоновом пути: нужно посетить каждый из n городов ровно один раз, при этом стартовать можно с любого города. Решается динамическим программированием по подмножествам.

Читаем матрицу расстояний из input.txt. Заводим массив, где для каждой пары «маска посещённых городов, последний город» храним минимальную длину пути. Дополнительно храним предыдущий город, чтобы потом восстановить маршрут.

Инициализация: для каждой маски из одного города значение равно нулю - любой город может быть стартовым. Далее перебираем все маски и все последние города в них. Если состояние достижимо, пробуем добавить непосещённый город: обновляем значение для новой маски через текущее значение плюс расстояние от последнего до нового, сохраняя родителя.

После обработки всех масок ищем минимум среди значений для полной маски по всем возможным последним городам - это оптимальная длина. Восстанавливаем путь, идя по

родителям от лучшего последнего города до стартового, и разворачиваем его.

Записываем длину и порядок городов в output.txt.

Сложность - $O(2^n \cdot n^2)$, при $n = 13$ это около 1,4 миллиона операций.

Описание проведённых тестов

input.txt			
1	5		
2	0 183 163 173 181		
3	183 0 165 172 171		
4	163 165 0 189 302		
5	173 172 189 0 167		
6	181 171 302 167 0		
7			

output.txt	
1	666
2	4 5 2 3 1

```
18 def test_minimal():
19     output = check(task_num: 16, input_data: "1\n0\n")
20     length, path = _parse(output)
21     assert length == 0
22     assert path == [1]
23
24
25 def test_example():
26     input_data = (
27         "5\n"
28         "0 183 163 173 181\n"
29         "183 0 165 172 171\n"
30         "163 165 0 189 302\n"
31         "173 172 189 0 167\n"
32         "181 171 302 167 0\n"
33     )
34     output = check(task_num: 16, input_data)
35     length, path = _parse(output)
36     assert length == 666
37     assert sorted(path) == [1, 2, 3, 4, 5]
```

```

40 def test_maximal():  Ⓜ MarieeMorte
41     random.seed(16)
42     n = 13
43     mat = [[0] * n for _ in range(n)]
44     for i in range(n):
45         for j in range(i + 1, n):
46             d = random.randint(a: 1, 10 ** 6)
47             mat[i][j] = d
48             mat[j][i] = d
49
50     lines = [str(n)] + [" ".join(map(str, row)) for row in mat]
51     input_data = "\n".join(lines) + "\n"
52
53     output = check(task_num: 16, input_data, timeout=60)
54     length, path = _parse(output)
55
56     assert sorted(path) == list(range(1, n + 1))
57     assert _path_length(mat, path) == length
58     assert length == _dp_reference(mat, n)

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.081s	memory=28.0 KB
Пример из задачи	time=0.114s	memory=140.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.189s	memory=2.4 MB

Вывод по задаче

Был реализован алгоритм динамического программирования по подмножествам для поиска кратчайшего гамильтонова пути. Старт разрешён с любого города, маршрут восстанавливается через массив родителей.

19 задача. Произведение матриц (3 балла, с учётом штрафа 2,5 балла)

Описание задания к задаче

- **Постановка задачи.** В произведении последовательности матриц полностью расставлены скобки, если выполняется один из следующих пунктов:
 - Произведение состоит из одной матрицы.
 - Оно является заключенным в скобки произведением двух произведений с полностью расставленными скобками.

Полная расстановка скобок называется оптимальной, если количество операций, требуемых для вычисления произведения, минимально.

Требуется найти оптимальную расстановку скобок в произведении последовательности матриц.

- **Формат ввода / входного файла (input.txt).** В первой строке входных данных содержится целое число n - количество матриц. В n следующих строк содержится по два целых числа a_i и b_i - количество строк и столбцов в i -той матрице соответственно. Гарантируется, что $b_i = a_{i+1}$ для любого $1 \leq i \leq n - 1$.
- **Ограничения на входные данные.** $1 \leq n \leq 400$, $1 \leq a_i, b_i \leq 100$ для всех $1 \leq i \leq n$.
- **Формат вывода / выходного файла (output.txt).** Выведите оптимальную расстановку скобок. Если таких расстановок несколько, выведите любую.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 64 мб.

Исходный код и описание решения

```
def solve_matrix_chain():
    with open('input.txt', 'r') as f:
        n = int(f.readline().strip())

        dimensions = []
        for _ in range(n):
            rows, cols = map(int, f.readline().strip().split())
            dimensions.append((rows, cols))

        p = []
        for i in range(n):
            p.append(dimensions[i][0])
        p.append(dimensions[-1][1])

        dp = [[0] * n for _ in range(n)]
        split = [[-1] * n for _ in range(n)]

        for length in range(2, n + 1):
```

```

        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = 10 ** 18

            for mid in range(i, j):
                cost = dp[i][mid] + dp[mid + 1][j] + p[i] * p[mid + 1] *
p[j + 1]

                if cost < dp[i][j]:
                    dp[i][j] = cost
                    split[i][j] = mid

def build_parentheses(left_idx, right_idx):
    if left_idx == right_idx:
        return "A"

    best_split = split[left_idx][right_idx]
    left = build_parentheses(left_idx, best_split)
    right = build_parentheses(best_split + 1, right_idx)

    return "(" + left + right + ")"

result = build_parentheses(0, n - 1)

with open('output.txt', 'w') as f:
    f.write(result)

if __name__ == "__main__":
    solve_matrix_chain()

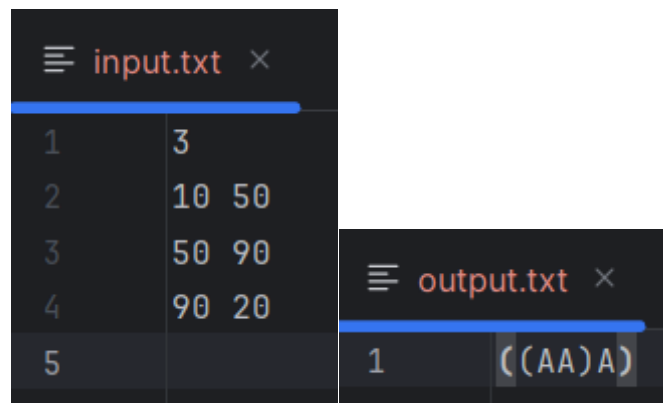
```

Сначала читаем из input.txt количество матриц и их размеры. Строим массив размеров: сначала число строк первой матрицы, затем число столбцов каждой матрицы по порядку. Благодаря гарантии, что число столбцов i -й матрицы равно числу строк $(i+1)$ -й, размеры согласованы, и стоимость перемножения двух матриц на стыке вычисляется как произведение трёх чисел.

Динамическое программирование: для каждого диапазона от i до j храним минимальную стоимость перемножения матриц в этом диапазоне и позицию разбиения, дающую эту стоимость. Перебираем длину подцепочки от 2 до n , для каждой пары (i, j) перебираем все позиции разбиения k от i до $j-1$ и пересчитываем стоимость как сумму стоимостей левой и правой частей плюс стоимость перемножения результатов. Запоминаем минимум и соответствующую позицию.

После заполнения таблиц восстанавливаем расстановку скобок рекурсивно: для диапазона из одной матрицы возвращаем «A», иначе рекурсивно строим левую и правую части через сохранённую позицию разбиения и оборачиваем их в скобки. Полученную строку записываем в output.txt.

Описание проведённых тестов



```
58 > def test_minimal(): & MarieeMorte
59     check( task_num: 19, input_data: "1\n10 20\n", expected: "A")
60
61
62 > def test_example(): & MarieeMorte
63     check( task_num: 19, input_data: "3\n10 50\n50 90\n90 20\n", expected: "((AA)A)")
64
65
66 > def test_maximal(): & MarieeMorte
67     n = 400
68     dims = [(100, 100)] * n
69     output = check( task_num: 19, _make_input(dims), timeout=60)
70     _check_structure(output, n)
```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.077s	memory=4.0 KB
Пример из задачи	time=0.163s	memory=96.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=1.880s	memory=4.5 MB

Вывод по задаче

Был реализован алгоритм динамического программирования для оптимальной расстановки скобок в цепочке матриц. Таблица разбиений позволяет не только найти минимальную стоимость, но и восстановить саму расстановку.

22 задача. Симпатичные узоры (4 балла, с учётом штрафа 3,5 балла)

Описание задания к задаче

- **Постановка задачи.** Компания BrokenTiles планирует заняться выкладыванием во дворах у состоятельных клиентов узор из черных и белых плиток, каждая из которых имеет размер 1×1 метр. Известно, что дворы всех состоятельных людей имеют наиболее модную на сегодня форму прямоугольника $M \times N$ метров. Однако при составлении финансового плана у директора этой организации появилось целых две серьезных проблемы: во первых, каждый новый клиент очевидно захочет, чтобы узор, выложенный у него во дворе, отличался от узоров всех остальных клиентов этой фирмы, а во вторых, этот узор должен быть симпатичным. Как показало исследование, узор является симпатичным, если в нем нигде не встречается квадрата 2×2 метра, полностью покрытого плитками одного цвета.
Для составления финансового плана директору необходимо узнать, сколько клиентов он сможет обслужить, прежде чем симпатичные узоры данного размера закончатся. Помогите ему!
- **Формат входного файла (input.txt).** В первой строке входного файла находятся два положительных целых числа, разделенные пробелом – M и N .
- **Ограничения на входные данные.** $1 \leq N \times M \leq 30$.
- **Формат выходного файла (output.txt).** Выведите в выходной файл единственное число – количество различных симпатичных узоров, которые можно выложить во дворе размера $M \times N$. Узоры, получающиеся друг из друга сдвигом, поворотом или отражением считаются различными.
- Ограничение по времени. 1 сек.
- Ограничение по памяти. 16 мб.

Исходный код и описание решения

```
def solve_pretty_patterns():
    with open('input.txt', 'r') as f:
        m, n = map(int, f.readline().strip().split())

    if m < n:
        height, width = n, m
    else:
        height, width = m, n
```

```

total_masks = 1 << width

compatible_masks = [[] for _ in range(total_masks)]

for mask1 in range(total_masks):
    for mask2 in range(total_masks):
        compatible = True
        for col in range(width - 1):
            if ((mask1 >> col) & 1) == ((mask1 >> (col + 1)) & 1) == \
                ((mask2 >> col) & 1) == ((mask2 >> (col + 1)) & 1):
                compatible = False
                break

        if compatible:
            compatible_masks[mask1].append(mask2)

dp = [1] * total_masks

for _ in range(1, height):
    new_dp = [0] * total_masks
    for mask1 in range(total_masks):
        if dp[mask1] == 0:
            continue
        for mask2 in compatible_masks[mask1]:
            new_dp[mask2] += dp[mask1]
    dp = new_dp

result = sum(dp)

with open('output.txt', 'w') as f:
    f.write(str(result))

if __name__ == "__main__":
    solve_pretty_patterns()

```

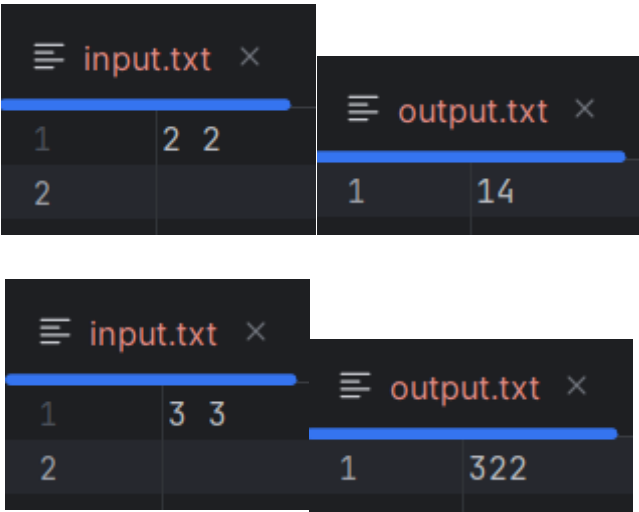
Так как произведение $M \cdot N$ не превышает 30, можно выбрать меньшую сторону как «ширину» - тогда состояний будет не больше $2^5 = 32$. Каждую строку узора кодируем битовой маской длины «ширина»: 0 - один цвет плитки, 1 — другой.

Сначала строим таблицу совместимости: для каждой пары масок проверяем, что ни в одной паре соседних столбцов не образуется одноцветный квадрат 2×2 . Если все четыре бита (по два в каждой строке) равны - маски несовместимы. Для каждой маски сохраняем список совместимых с ней - так переходы в динамическом программировании станут быстрее.

Динамическое программирование по строкам: массив, в котором для каждой маски храним количество узоров, у которых последняя рассмотренная строка имеет такой вид. Изначально для всех масок значение равно единице (первая строка любая). Для каждой следующей строки строим новый массив: для каждой маски и каждой совместимой с ней добавляем соответствующее значение. После обработки всех строк суммируем по всем маскам - это ответ.

Записываем его в output.txt. Сложность - $O(\text{высота} \cdot 2^{\text{ширина}} \cdot \text{ширина})$, при $M \cdot N \leq 30$ это порядка 150 тысяч операций.

Описание проведённых тестов



ID	Дата	Автор	Задача	Язык	Результат	Тест	Время	Память
25417437	09.09.2026 18:37:43	Борзова Дарья Максимовна	0083	Python	Accepted		0,062	316 Кб

```
24 ▶ def test_minimal(): & MarieeMorte
25     check( task_num: 22, input_data: "1 1\n", expected: "2")
26
27
28 ▶ def test_example_1(): & MarieeMorte
29     check( task_num: 22, input_data: "2 2\n", expected: "14")
30
31
32 ▶ def test_example_2(): & MarieeMorte
33     check( task_num: 22, input_data: "3 3\n", expected: "322")
34
35
36 ▶ def test_maximal(): & MarieeMorte
37     check( task_num: 22, input_data: "1 30\n", str(2 ** 30), timeout=30)
```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.077s	memory=92.0 KB
Первый пример из задачи	time=0.225s	memory=256.0 KB
Второй пример из задачи	time=0.231s	memory=220.0 KB
Верхняя граница диапазона значений	time=0.257s	memory=328.0 KB

ВХОДНЫХ ДАННЫХ ИЗ ТЕКСТА ЗАДАЧИ		
---------------------------------	--	--

Вывод по задаче

Был реализован алгоритм динамического программирования по профилю с кодированием строки битовой маской. Выбор меньшей стороны прямоугольника в качестве ширины минимизирует число состояний, предварительная таблица совместимости ускоряет переходы.

Вывод по проделанной работе

В ходе выполнения лабораторной работы были изучены и реализованы жадные алгоритмы и различные вариации динамического программирования: линейное, интервальное, по подмножествам и по профилю. Жадный подход оказался эффективен в задачах о дробном рюкзаке, максимизации дохода от рекламы и расписании лекций. Динамическое программирование применялось для задач о рюкзаке 0/1, разделении сувениров, максимизации арифметического выражения, поиске кратчайшего гамильтонова пути, подсчёте телефонных номеров, расстановке скобок в произведении матриц и подсчёте узоров без одноцветных квадратов. Все задачи решены на языке Python и укладываются в заданные ограничения по времени и памяти.